# Listas Duplamente Encadeadas

**Professor:** André Felipe Ribeiro Cordeiro
**Disciplina:** Estruturas de Dados

**Departamento de Informática**
**Centro de Tecnologia**
**Universidade Estadual de Maringá**

# Introdução

- As **estruturas de lista**, estudadas até o momento, caracterizam-se por formar um **encadeamento simples** entre os elementos.

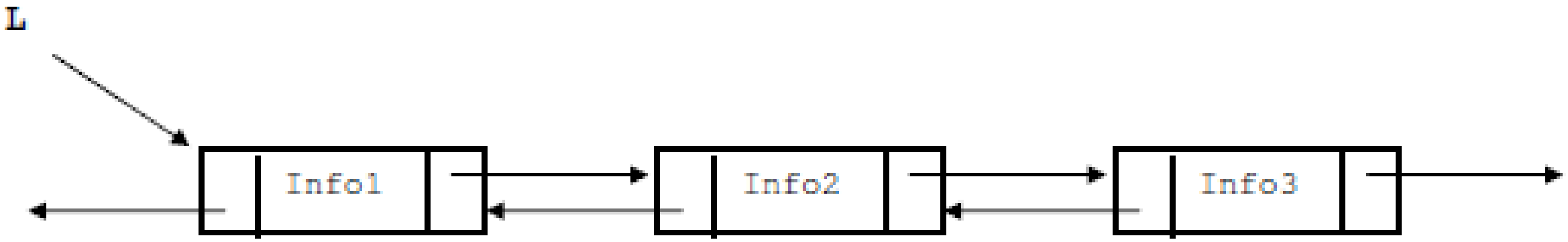
- Cada elemento armazena um **ponteiro** para o **próximo elemento** da lista.

# Listas
# Duplamente Encadeadas

- **Listas duplamente encadeadas** apresentam uma organização diferente.

- Nessas listas, cada elemento possui **dois ponteiros**. Um para referenciar o **próximo elemento** e outro para referenciar o **elemento anterior**.

# Representação



# Operações
# Listas Duplamente Encadeadas

- As seguintes operações podem ser utilizadas em diferentes aplicações:
  - **inicializar** a lista
  - **verificar** se a lista está **vazia**
  - **inserir item** na lista
  - **retirar item** da lista
  - **imprimir os itens** da lista

# Implementação

```
#include <stdlib.h>
#include <stdio.h>
```

```
typedef struct TipoItem *TipoApontador;

typedef struct TipoItem {
  TipoApontador anterior;
  int conteudo;
  TipoApontador proximo;
} TipoItem;
```

# Implementação

```
typedef struct {
  TipoApontador primeiro, ultimo;
} TipoLista;
```

```
int Vazia(TipoLista lista){
  return (lista.primeiro == lista.ultimo);
}
```

# Implementação

```
void Inicializa(TipoLista *lista){
  lista->primeiro =
     (TipoApontador) malloc (sizeof(TipoItem));
  lista->primeiro->anterior = NULL;
  lista->primeiro->proximo = NULL;
  lista->ultimo = lista->primeiro;
}
```

# Implementação

```
void Insere(int elemento, TipoLista *lista){
  lista->ultimo->proximo =
     (TipoApontador) malloc (sizeof(TipoItem));
  lista->ultimo->proximo->anterior =
                          lista->ultimo->proximo;

  lista->ultimo = lista->ultimo->proximo;
  lista->ultimo->conteudo = elemento;
  lista->ultimo->proximo = NULL;
}
```

# Implementação

```
void Retira(TipoApontador ponteiro,
                                TipoLista *lista, int elemento){
  TipoApontador p_auxiliar;
  if (Vazia(*lista)
      || ponteiro == NULL || ponteiro->proximo == NULL) {
    printf(" Erro! Lista vazia ou posicao nao existe \n");
    return;
  }

  p_auxiliar = ponteiro->proximo;
  elemento = p_auxiliar->conteudo;
  .....
}
```

# Implementação

```
.....
  if (p_auxiliar->proximo != NULL){
    p_auxiliar->proximo->anterior = p_auxiliar->anterior;
    ponteiro->proximo = p_auxiliar->proximo;
  }else{
    lista->ultimo = p_auxiliar->anterior;
    lista->ultimo->proximo = NULL;
    ponteiro->proximo = NULL;
  }
  free(p_auxiliar);
}
```

# Implementação

```
void Imprime(TipoLista lista){
  TipoApontador p_auxiliar;
  p_auxiliar = lista.primeiro->proximo;

  while (p_auxiliar != NULL) {
    printf("%d\n", p_auxiliar->conteudo);
    p_auxiliar = p_auxiliar->proximo;
  }
}
```

# Observações

- Outras operações poderiam ser consideradas, além das apresentadas.

- **Exemplos**
  - **Inserção** à esquerda na lista
  - **Elemento** à direita na lista

# Referências

- Introdução a Estruturas de Dados, Waldemar Celes, Renato Cerqueira e José Rangel, 2016.

- Projeto de Algoritmos com implementações em Pascal e C, Nivio Ziviani, 2011.

- Aulas Professor Rodolfo Miranda Pereira, UEM.